Manual UC00606 – Desenvolver programas em linguagem estruturada

Programar em C/C++

Formador: Luis Santos

2026

Índice

Antes de começar.....3 1. O Code::Blocks.....4 2. Estrutura de um programa .....5 3. Variáveis, tipos e constantes.....6 4. Entrada e saída: printf/scanf e cout/cin.....7 5. Operadores e expressões.....9 6. Estruturas de decisão: if e switch.....10 7. Estruturas de repetição: for, while e do-while.....12 8. Vetores e texto .....14 9. Funções e procedimentos .....16 10. Erros e depuração.....18 11. Exercícios.....19 12. Cábula final.....20

Antes de começar

Este manual é para ter ao lado do teclado. Não é para ler de uma ponta à outra: é para consultar quando não se lembrarem de como se escreve um `for` ou porque é que o `scanf` não leu nada.

Nesta UFCD (UC00606, 4,5 pontos de crédito) o objetivo é simples: pegar nos algoritmos que já sabem fazer em pseudocódigo e passá-los para código que compila e funciona. No fim, têm de conseguir:

- planear um programa antes de o escrever;
- escrever programas com variáveis, decisões, ciclos e funções;
- testar, encontrar os erros e corrigi-los.

Vamos trabalhar em **C**, no **Code::Blocks**. Uma nota importante: o ficheiro vai ser `.cpp`, porque vamos usar lado a lado o `printf`/`scanf` do C e o `cout`/`cin` do C++. Quero que se habituem aos dois. Tudo o resto (variáveis, `if`, ciclos, funções) escreve-se da mesma maneira nas duas linguagens.

Um conselho que vale mais do que o manual todo: **escrevam o código à mão, não copiem e colem**. Os dedos também aprendem.

1. O Code::Blocks

O Code::Blocks é o nosso ambiente de trabalho (IDE): editor, compilador e depurador no mesmo sítio. Na instalação, escolham sempre a versão que diz **mingw-setup** - é a que já traz o compilador. Sem ela, o programa abre mas não compila nada.

Criar um projeto novo

1. File → New → Project → **Console application** → Go.
2. Escolham **C++** (sim, C++, pelas razões que já expliquei).
3. Dêem um nome ao projeto sem espaços nem acentos (ex.: `ficha01_ex3`) e escolham uma pasta vossa.
4. Next → Finish. Do lado esquerdo, em *Sources*, aparece o `main.cpp`. É aí que se trabalha.

Compilar e correr

Tecla	O que faz

F9	Compila e corre (é a que vão usar 90% das vezes)
Ctrl + F9	Só compila
Ctrl + F10	Só corre a última versão compilada
F5	Põe/tira um ponto de paragem (*breakpoint*) na linha
F8	Começa a depuração

O programa abre numa janela preta, a **consola**. Se houver erros, aparecem em baixo, no separador *Build messages*, com o número da linha. Façam duplo clique no erro e o cursor salta para lá.

Dois hábitos a ganhar desde o primeiro dia: gravar com Ctrl + S antes de compilar e manter o código indentado (Plugins → Source code formatter, se ficar tudo torto).

2. Estrutura de um programa

Todos os programas que vamos fazer começam com este esqueleto. Decorem-no.

```c
#include <stdio.h> // printf e scanf (C)
#include <iostream> // cout e cin (C++)

using namespace std; // para escrever cout em vez de std::cout

int main()
{
 printf("Ola, mundo!\n");
 cout << "Ola outra vez!" << endl;

 return 0;
}
```

Linha a linha:

- `#include` - vai buscar bibliotecas já feitas. Sem o `stdio.h` o `printf` não existe; sem o `iostream` o `cout` não existe.
- `using namespace std;` - só serve para não termos de escrever `std::` antes do `cout`, `cin` e `endl`.
- `int main()` - é aqui que o programa começa. Há sempre um, e só um, `main`. •
`{ }` - as chavetas marcam o início e o fim de um bloco.
- `return 0;` - diz ao sistema operativo que o programa terminou bem.

Regras que dão erro se falharem:

- Cada instrução acaba em **ponto e vírgula** `;`. É o erro número um.
- O C distingue maiúsculas de minúsculas: `Idade`, `idade` e `IDADE` são três coisas diferentes. •
Cada `{` tem de ter o seu `}`, cada `(` o seu `)`.

Comentários - o compilador ignora-os, mas quem lê o código agradece:

```c
// comentário de uma linha
/* comentário
 de várias linhas */
```

Um aviso: nas mensagens para o ecrã não se admirem se os acentos e o ç aparecerem como símbolos estranhos. É a consola do Windows. A solução está no capítulo 10.

3. Variáveis, tipos e constantes

Uma variável é uma caixa na memória com um nome e um tipo. O tipo diz o que pode lá ir dentro. Em C, **declara-se sempre antes de usar**.

```c
int idade; // declarar
idade = 17; // atribuir
float media = 14.5; // declarar e atribuir de uma vez
int a, b, c; // várias do mesmo tipo
```

Os tipos que vamos usar

Tipo	Guarda	Exemplo	printf/scanf
`int`	números inteiros	`int n = -5;`	`%d`
`float`	números com casas decimais	`float preco = 2.99;`	`%f`
`double`	decimais com mais precisão	`double pi = 3.14159265;`	`%lf`
`char`	um único carácter	`char letra = 'A';`	`%c`
`bool`	verdadeiro/falso (C++)	`bool fim = false;`	-

Atenção a dois pormenores que tiram pontos nos testes:

- O separador decimal no código é o **ponto**, não a vírgula: `2.99` e não `2,99`.
- Um `char` leva **plicas** (`'A'`); um texto leva **aspas** (`"Ana"`).

Regras para os nomes: começam por letra ou `_`, sem espaços, sem acentos, sem palavras reservadas (`int`, `for`, `if`…). Usem nomes que digam o que a variável guarda: `total` é melhor que `t`, `numAlunos` é melhor que `x`.

Constantes - valores que não mudam durante o programa:

```c
#define IVA 0.23 // à moda do C, sem ; no fim
const int MAX_ALUNOS = 30; // à moda do C++, com tipo
```

Por convenção, as constantes escrevem-se em MAIÚSCULAS. Se tentarem alterar uma `const`, o compilador não deixa - e ainda bem.

4. Entrada e saída: printf/scanf e cout/cin

Fazem exatamente o mesmo; muda a forma de escrever. Vejam o mesmo programa das duas maneiras:

```
// ---- À moda do C ----
int idade;
printf("Qual a tua idade? ");
scanf("%d", &idade);
printf("Tens %d anos.\n", idade);

// ---- À moda do C++ ----
int idade2;
cout << "Qual a tua idade? ";
cin >> idade2;
cout << "Tens " << idade2 << " anos." << endl;
```

printf e scanf

O `printf` escreve o texto tal como está e troca cada `%` pelo valor da variável, pela ordem em que aparecem.

```
float nota = 15.456;
printf("Nome: %s | Nota: %.1f\n", "Rui", nota); // Nome: Rui | Nota: 15.5
```

- `%.2f` - mostra 2 casas decimais (e arredonda).
- `%5d` - ocupa 5 espaços, útil para alinhar tabelas.
- `\n` muda de linha, `\t` dá um *tab*, `%%` escreve o símbolo %.

O `scanf` lê do teclado. **Leva sempre o `&` antes da variável** (a exceção são os textos, que vemos no capítulo 8). Esquecer o `&` não dá erro a compilar, mas o programa rebenta ou lê lixo.

```
int a; float b;
scanf("%d %f", &a, &b); // lê as duas de seguida
```

cout e cin

O `cout` usa `<<` ("manda para o ecrã") e o `cin` usa `>>` ("tira do teclado para a variável"). Não precisam de `%d` nem de `&`: o C++ percebe sozinho o tipo.

```
int a; float b;
cin >> a >> b;
cout << "Soma: " << a + b << endl; // endl = mudar de linha
```

Para casas decimais no `cout`:

```
#include <iomanip>
cout << fixed << setprecision(2) << b << endl; // 2 casas decimais
```

Qual usar?

Situação	Mais prático
Mostrar texto simples	os dois são iguais
Controlar casas decimais e alinhamento	`printf`

Situação	Mais prático
Ler sem se preocupar com o tipo	`cin`
Testes e fichas desta UFCD	o que o enunciado pedir

Podem misturar os dois no mesmo programa. O que não devem é esquecer-se que o `scanf` tem `&` e o `cin` não.



5. Operadores e expressões

Aritméticos

Operador	Faz	Exemplo	Resultado

`+ - *`	soma, subtração, multiplicação	`4 * 3`	`12`
`/`	divisão	`7 / 2`	`3` (!)
`%`	resto da divisão inteira	`7 % 2`	`1`

O `7 / 2` dá `3` porque os dois números são inteiros e o C corta as casas decimais. Para dar `3.5`, pelo menos um deles tem de ser decimal: `7.0 / 2` ou `(float)a / b`. Este é talvez o erro que mais vezes vou corrigir nas vossas fichas.

O `%` é muito útil: `n % 2 == 0` diz se `n` é par; `n % 10` dá o último algarismo.

Relacionais (dão verdadeiro ou falso): `==` igual, `!=` diferente, `>`, `<`, `>=`, `<=`.

Cuidado: `=` **atribui**, `==` **compara**. `if (x = 5)` compila, mas está errado e é sempre

verdadeiro. **Lógicos**

Operador	Lê-se	Exemplo
`&&`	E	`nota >= 10 && nota <= 20`
`||`	OU	`op == 's' || op == 'S'`
`!`	NÃO	`!(idade < 18)`

Atalhos que vão ver em todo o lado

```
i++; // i = i + 1
i--; // i = i - 1
total += x; // total = total + x
n *= 2; // n = n * 2
```

Funções matemáticas - precisam de `#include <math.h>` (ou `<cmath>`): `sqrt(x)` raiz quadrada, `pow(b, e)` potência, `abs(n)` valor absoluto.

Quando tiverem dúvidas na ordem das contas, ponham parênteses. `(a + b) / 2` não é o mesmo que `a + b / 2`.



6. Estruturas de decisão: if e switch

É o **Se… Então… Senão** do pseudocódigo.

```
if (nota >= 10)
{
 printf("Aprovado\n");
}
else
{
 printf("Reprovado\n");
}
```

Quando há mais de dois caminhos, encadeia-se com `else if`:

```
if (nota < 0 || nota > 20)
 cout << "Nota invalida" << endl;
else if (nota < 10)
 cout << "Insuficiente" << endl;
else if (nota < 14)
 cout << "Suficiente" << endl;
else
 cout << "Bom" << endl;
```

Se dentro do `if` houver só uma instrução, as chavetas podem ficar de fora. O meu conselho: ponham-nas sempre enquanto estão a aprender. Poupam-vos muitos erros quando acrescentarem uma segunda linha.

E nunca ponham `;` logo a seguir ao `if (...)`. O `if` fica vazio e o bloco de baixo corre sempre.

switch

O `switch` é o **Caso** do pseudocódigo. Serve quando comparamos **uma** variável `int` ou `char` com vários valores exatos - perfeito para menus.

```
int opcao;
printf("1 - Somar\n2 - Subtrair\n0 - Sair\nOpcao: ");
scanf("%d", &opcao);

switch (opcao)
{
 case 1:
 printf("Escolheu somar\n");
 break;
 case 2:
 printf("Escolheu subtrair\n");
 break;
 case 0:
 printf("Adeus!\n");
 break;
 default:
 printf("Opcao invalida\n");
}
```

- Cada `case` termina com `break`. Sem ele, o programa continua a executar o `case` seguinte.



- O `default` é o “em qualquer outro caso”.
- O `switch` não funciona com intervalos (`nota > 10`) nem com `float`. Para isso, usem `if`.



7. Estruturas de repetição: for, while e do-while

Três ciclos, três situações. A pergunta a fazer é: **sei quantas vezes vai repetir?**

Ciclo	Pseudocódigo	Quando usar
`for`	Para… de… até…	sei quantas vezes
`while`	Enquanto… faz	não sei; pode nem executar nenhuma vez
`do...while`	Repete… até	não sei; executa pelo menos uma vez

for

```
for (int i = 1; i <= 10; i++)
{
 printf("%d x 7 = %d\n", i, i * 7);
}
```

Dentro dos parênteses há três partes, separadas por `;`: **início** (`int i = 1`), **condição para continuar** (`i <= 10`) e **o que muda em cada volta** (`i++`).

while

Testa a condição **antes** de entrar. Se for falsa logo à primeira, o bloco nunca corre.

```
int num;
cout << "Numero (0 para terminar): ";
cin >> num;
while (num != 0)
{
 cout << "O dobro e " << num * 2 << endl;
 cout << "Numero (0 para terminar): ";
 cin >> num;
}
```

do…while

Testa **depois**, por isso corre sempre pelo menos uma vez. É o ideal para validar dados e para menus.

```
int nota;
do
{
 printf("Nota (0 a 20): ");
 scanf("%d", &nota);
} while (nota < 0 || nota > 20); // atenção ao ; no fim
```

Contadores e acumuladores

São os mesmos do pseudocódigo. Inicializem-nos sempre a zero **antes** do ciclo - uma variável por inicializar tem lixo lá dentro.

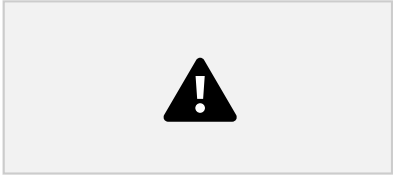
```
int conta = 0;
float soma = 0;
for (int i = 0; i < 5; i++)
{
 float valor;
```



```
scanf("%f", &valor);
 soma += valor; // acumulador
 conta++; // contador
}
printf("Media: %.2f\n", soma / conta);
```

Ciclo infinito? Se a condição nunca ficar falsa, o programa nunca termina. Fechem a consola (ou Ctrl + C) e procurem a variável que se esqueceram de alterar dentro do ciclo.

Dentro de um ciclo, `break` sai do ciclo de imediato e `continue` salta para a volta seguinte. Usem com moderação.



8. Vetores e texto

Um vetor (ou *array*) é uma fila de variáveis do mesmo tipo com um só nome. Em vez de `nota1`, `nota2`… `nota30`, temos `nota[30]`.

```
int nota[5]; // 5 inteiros: nota[0] a nota[4] float preco[3] =
{1.5, 2.0, 0.9}; // declarar já com valores
```

A regra de ouro: **as posições começam em 0**. Um vetor de 5 vai de `[0]` a `[4]`. Escrever em `nota[5]` não dá erro a compilar, mas estraga memória que não é vossa.

Os vetores e o `for` andam sempre juntos:

```c
#define N 5
int nota[N], soma = 0;

for (int i = 0; i < N; i++)
{
 printf("Nota %d: ", i + 1);
 scanf("%d", &nota[i]);
 soma += nota[i];
}
printf("Media: %.1f\n", (float)soma / N);
```

Texto (strings)

Em C, um texto é um vetor de `char` que termina num carácter invisível, o `'\0'`. Por isso reservem sempre uma posição a mais.

```c
#include <string.h>

char nome[31]; // até 30 letras + o '\0'

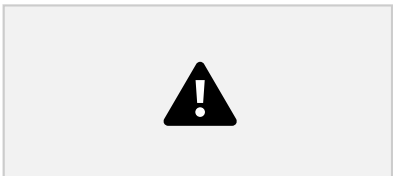
printf("Nome: ");
scanf("%30s", nome); // SEM & - o nome do vetor já é um endereço
printf("Ola, %s!\n", nome);
```

O `scanf("%s")` para de ler no primeiro espaço: se escreverem "Ana Silva", só guarda "Ana". Para ler uma linha inteira:

```c
fgets(nome, 31, stdin); // à moda do C
cin.getline(nome, 31); // à moda do C++
```

Funções úteis do `string.h`:

Função	Faz
`strlen(s)`	número de caracteres
`strcpy(dest, orig)`	copia um texto para outro (não se usa `=`)
`strcmp(a, b)`	compara; dá `0` se forem iguais (não se usa `==`)
`strcat(dest, orig)`	junta `orig` ao fim de `dest`



Cuidado ao misturar números e texto: depois de um `scanf("%d")` ou `cin >> n`, o Enter fica no teclado e o `fgets`/`getline` seguinte lê uma linha vazia. Resolve-se limpando antes: `getchar();` em C ou

`cin.ignore();` em C++.



9. Funções e procedimentos

Quando o `main` começa a ficar comprido, parte-se o problema em pedaços. Cada pedaço é uma função: faz uma coisa, tem um nome e pode ser chamada as vezes que quisermos.

- **Função** - faz contas e **devolve** um valor com `return`.
- **Procedimento** - faz qualquer coisa (mostrar um menu, por exemplo) e não devolve nada. Em C é uma função do tipo `void`.

```
#include <stdio.h>
#include <iostream>
using namespace std;

// 1) Protótipos: avisam o compilador de que as funções existem
void mostrarMenu();
float media(float a, float b);

int main()
{
 mostrarMenu(); // chamar um procedimento  float m = media(12, 15); // chamar uma
função e guardar o resultado  printf("Media: %.1f\n", m);
 return 0;
}

// 2) Definições: o código de cada uma
void mostrarMenu()
{
 cout << "1 - Calcular media" << endl;
 cout << "0 - Sair" << endl;
}

float media(float a, float b)
{
 return (a + b) / 2;
}
```

A primeira linha de uma função diz tudo: **tipo que devolve**, **nome** e **parâmetros** (o que precisa de receber). Se escreverem as funções todas antes do `main`, não precisam dos protótipos.

Variáveis locais e globais

- **Local** - declarada dentro de uma função. Só existe lá dentro e desaparece quando a função termina. Duas funções podem ter uma variável `i` cada uma sem se chatearem.
- **Global** - declarada fora de todas as funções, lá em cima. Todas a veem e todas a podem alterar.

Usem locais sempre que possível. Com globais, quando algo corre mal, é muito difícil descobrir quem mexeu no valor.

Passagem de parâmetros

Por valor (a normal): a função recebe uma **cópia**. Pode alterá-la à vontade que a variável original não muda.

Por referência: a função recebe a própria variável e as alterações ficam. Em C++ basta pôr um `&` no parâmetro:



```
void trocar(int &x, int &y)
{
 int aux = x;
 x = y;
 y = aux;
}

// no main:
int a = 3, b = 8;
trocar(a, b); // agora a = 8 e b = 3
```

Sem o `&` na declaração, a troca acontecia só nas cópias e no `main` ficava tudo igual. Experimentem tirar o `&` e vejam.

Os vetores são sempre passados por referência: `void lerNotas(int v[], int n)` altera mesmo o vetor do `main`.



10. Erros e depuração

Errar faz parte. O que distingue um programador é a rapidez com que encontra o erro. Há dois tipos:

- **Erros de compilação** - o código está mal escrito e nem chega a correr. O Code::Blocks mostra-os a vermelho, com a linha.
- **Erros lógicos** - compila e corre, mas o resultado está errado. Estes só se apanham a testar.

As mensagens que mais vão ver

Mensagem	O que costuma ser
`expected ';' before ...`	falta um `;` na linha **anterior** à indicada

`'x' was not declared in this scope`	variável não declarada ou nome mal escrito (maiúsculas!)
`expected '}' at end of input`	falta fechar uma chaveta
`'cout' was not declared`	falta o `#include <iostream>` ou o `using namespace std;`
`undefined reference to 'WinMain'`	o ficheiro não tem `main`, ou tem `Main`

Dica: resolvam sempre **o primeiro erro da lista** e compilem outra vez. Muitas vezes um erro gera dez a seguir.

Como encontrar um erro lógico

1. **Testem com valores que sabem o resultado.** Se a média de 10 e 20 não der 15, há erro. 2. **Testem os limites**: zero, números negativos, o primeiro e o último elemento do vetor. 3. **Espalhem `printf`** pelo código para ver o valor das variáveis a meio. É rudimentar, mas funciona. 4. **Usem o depurador**: F5 numa linha para pôr um *breakpoint*, F8 para arrancar, e depois Debug →

   Next line para avançar linha a linha. Em Debug → Debugging windows → Watches veem o valor de cada variável.

Acentos na consola

Para os acentos e o ç aparecerem bem no Windows, ponham isto no início do `main`:

```
#include <locale.h>
...
setlocale(LC_ALL, "Portuguese");
```

Funciona para o que se escreve no ecrã. Para o que se lê do teclado nem sempre resulta, por isso nos nomes de variáveis e nos dados de teste continuem sem acentos.

Boas regras de programação

- Uma instrução por linha e indentação certinha (4 espaços por nível).
- Nomes que se percebem.
- Um comentário no topo com o vosso nome, a data e o que o programa faz. •

Validar o que o utilizador escreve (com `do...while`).

- Gravar muitas vezes. E fazer cópias.



11. Exercícios

Façam cada exercício duas vezes: uma com `printf`/`scanf` e outra com `cout`/`cin`. Antes de abrir o Code::Blocks, escrevam em papel o que o programa recebe, o que calcula e o que mostra.

Entrada, saída e operadores (cap. 3 a 5)

1. Pedir o nome e a idade e responder "Ola Ana, daqui a 10 anos tens 27."
2. Ler o preço sem IVA de um artigo e mostrar o preço com IVA a 23%, com duas casas decimais. 3. Ler um número de segundos e mostrar em horas, minutos e segundos (usem `/` e `%`). **Decisão (cap. 6)**
4. Ler um número e dizer se é par ou ímpar.
5. Ler uma nota de 0 a 20 e mostrar Insuficiente, Suficiente, Bom ou Muito Bom. 6. Calculadora: ler dois números e um operador (`+ - * /`) e mostrar o resultado com `switch`. Não deixar dividir por zero.

Repetição (cap. 7)

7. Mostrar a tabuada de um número escolhido pelo utilizador.
8. Ler números até aparecer o 0 e no fim mostrar quantos foram lidos, a soma e a média. 9. Pedir uma nota até ela estar entre 0 e 20 (validação com `do…while`).

Vetores e texto (cap. 8)

10. Ler as notas de 10 alunos para um vetor e mostrar a média, a nota mais alta e quantos tiveram positiva.
11. Ler um nome e mostrá-lo ao contrário.

Funções (cap. 9)

12. Criar as funções `int maior(int a, int b)`, `bool ePar(int n)` e `void linha(int n)` (desenha `n` traços). Usá-las num `main`.
13. **Mini-projeto:** programa com menu (`do…while` + `switch`) para gerir as notas de uma turma: inserir notas, listar, mostrar média, mostrar melhor aluno, sair. Cada opção é uma função. Entregar o código e uma folha com as instruções de utilização.



12. Cábula final

Imprimam esta página e tenham-na sempre à mão.

Quero…	C	C++

incluir	`#include <stdio.h>`	`#include <iostream>` + `using namespace std;`
escrever texto	`printf("Ola\n");`	`cout << "Ola" << endl;`
escrever um inteiro	`printf("%d", n);`	`cout << n;`
escrever decimal (2 casas)	`printf("%.2f", x);`	`cout << fixed << setprecision(2) << x;`
ler um inteiro	`scanf("%d", &n);`	`cin >> n;`
ler um decimal	`scanf("%f", &x);`	`cin >> x;`
ler um carácter	`scanf(" %c", &c);`	`cin >> c;`
ler uma palavra	`scanf("%s", nome);`	`cin >> nome;`
ler uma linha	`fge stdin);cin.getlin`	

(Reparem no espaço antes do `%c`: faz o `scanf` ignorar o Enter que ficou para trás.)

```
// DECISÃO
if (cond) { ... } else if (cond2) { ... } else { ... }

switch (op) { case 1: ... break; default: ... }

// REPETIÇÃO
for (int i = 0; i < n; i++) { ... }
while (cond) { ... }
do { ... } while (cond);

// VETOR
int v[10]; // posições 0 a 9

// FUNÇÕES
int soma(int a, int b) { return a + b; } // devolve valor
void mostrar(int x) { printf("%d", x); } // procedimento
void dobrar(int &x) { x = x * 2; } // por referência
```

Os cinco erros clássicos: falta de `;` · `=` em vez de `==` · `scanf` sem `&` · divisão inteira (`7/2 = 3`) · vetor a começar em 1 em vez de 0.